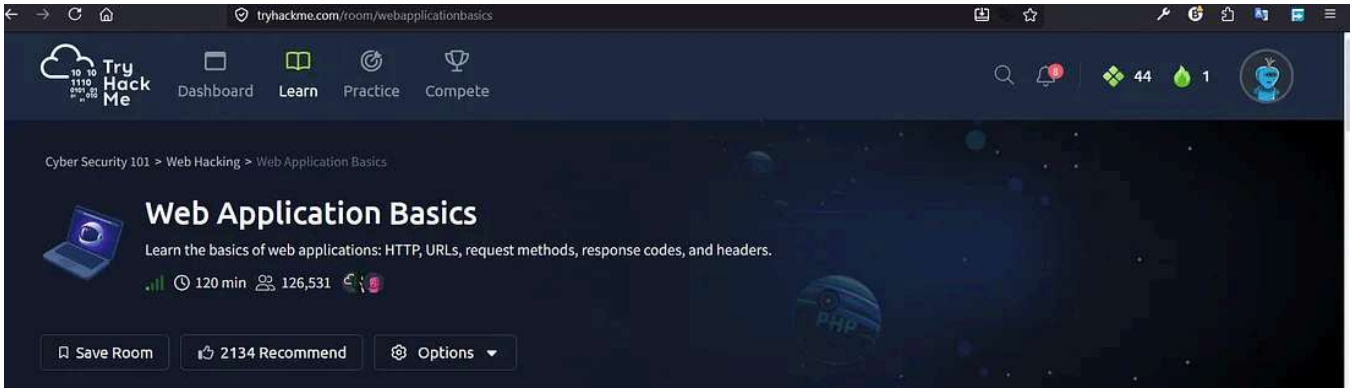


Web Applications +

Web Application Basic “ TRYHACKME “ Walkthrough

8 min read · Jun 3, 2026

Nuh hozar Follow

Listen Share

Web Application Basics introducing

Learn the basics of web applications: HTTP, URLs, request methods, response codes, and headers.

Cyber Security 101

- Web Hacking
- Web Application Basics
- *Tryhackme Walkthrough*
- Learn the basics of web applications: HTTP, URLs, request methods, response codes, and headers.
- Learning Objectives

By completing this room, you will:

- Understand what a web application is and how it runs in a web browser.
- Break down the components of a URL and see how it helps access web resources.
- Learn how requests and responses work.
- Get familiar with the different types of request methods.
- Understand what different response codes mean.
- Check out how headers work and why they matter for security.

Task 1 :- Introduction

Q1) I am ready to learn about Web Applications!
Answers :- N*o answer needed*

Task 2 :-Web Application Overview

Summary

There are many components involved in delivering a web application. Front End components like HTML, CSS, and JavaScript focus on the experience inside the browser. Back End components such as the Web Server, Database, or WAF are the engine under the surface that enable the web application to function. This simple introduction will be built upon in the upcoming tasks.

Q2) Which component on a computer is responsible for hosting and delivering content for web applications?
Answers :- w*eb server*

Q3) Which component on a computer is responsible for hosting and delivering content for web applications?
Answers :- *web browser*

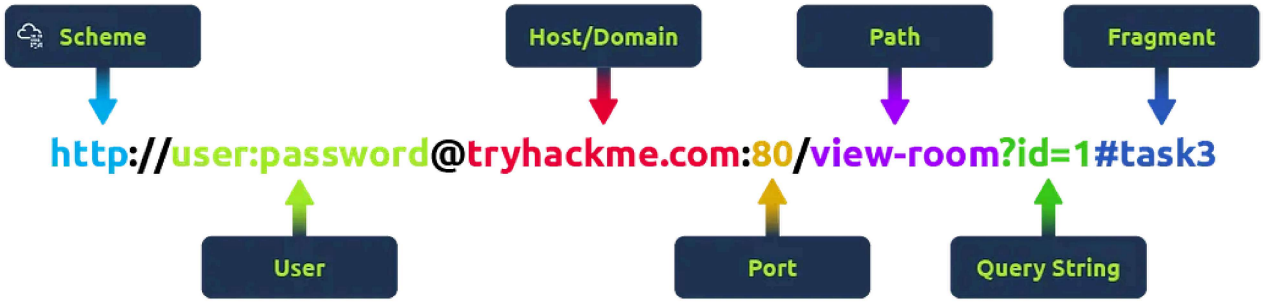
Q4) Which component acts as a protective layer, filtering incoming traffic to block malicious attacks, and ensuring the security of the the web application?
Answers :- *web application firewall*

Task 3 :-Uniform Resource Locator

Uniform Resource Locator

A Uniform Resource Locator (URL) is a web address that lets you access all kinds of online content — whether it's a webpage, a video, a photo, or other media. It guides your browser to the right place on the Internet.

Anatomy of a URL



Think of a URL as being made up of several parts, each playing a different role in helping you find the right resource. Understanding how these parts fit together is important for browsing the web, developing web applications, and even troubleshooting problems.

Here's a breakdown of the key components:

Scheme

The **scheme** is the protocol used to access the website. The most common are (HyperText Transfer Protocol) and **HTTPS** (Hypertext Transfer Protocol Secure). HTTPS is more secure because it encrypts the connection, which is why browsers and cyber security experts recommend it. Websites often enforce HTTPS for added protection.

User

Some URLs can include a user's login details (usually a username) for sites that require authentication. This happens mostly in URLs that need credentials to access certain resources. However, it's rare nowadays because putting login details in the URL isn't very safe — it can expose sensitive information, which is a security risk.

Host/Domain

The **host** or **domain** is the most important part of the URL because it tells you which website you're accessing. Every domain name has to be unique and is registered through domain registrars. From a security standpoint, look for domain names that

appear almost like real ones but have small differences (this is called). These fake domains are often used in attacks to trick people into giving up sensitive info.

Port

The **port number** helps direct your browser to the right service on the web server. It's like telling the server which doorway to use for communication. Port numbers range from 1 to 65,535, but the most common are **80** for and **443** for HTTPS.

Path

The **path** points to the specific file or page on the server that you're trying to access. It's like a roadmap that shows the browser where to go. Websites need to secure these paths to make sure only authorised users can access sensitive resources.

Query String

The **query string** is the part of the URL that starts with a question mark (?). It's often used for things like search terms or form inputs. Since users can modify these query strings, it's important to handle them securely to prevent attacks like **injections**, where malicious code could be added.

Fragment

The **fragment** starts with a hash symbol (#) and helps point to a specific section of a webpage — like jumping directly to a particular heading or table. Users can modify this too, so like with query strings, it's important to check and clean up any data here to avoid issues like injection attacks.

Q5) Which protocol provides encrypted communication to ensure secure data transmission between a web browser and a web server?
Answers :- *HTTPS*

Q6) What term describes the practice of registering domain names that are misspelt variations of popular websites to exploit user errors and potentially engage in fraudulent activities?
Answers :- *Typosquatting*

Q7) What part of a URL is used to pass additional information, such as search terms or form inputs, to the web server?

Answers :- *Query String*

Task 4 :-HTTP Messages

Http messages are packets of data exchanged between a user (the client) and the web server. These messages are very important for understanding how web applications work because they show how users' requests and the server's responses are communicated.

Imagine an example of an Request and an Response, where you can see key parts like the method, URL, headers, and status codes. These are what make the client-server interaction possible.

There are two types of messages:

- **Requests**: Sent by the user to trigger actions on the web application.
- **Responses**: Sent by the server in response to the user's request.

Q8) Which HTTP message is returned by the web server after processing a client's request?
Answers :- *HTTP response*

Q9) What follows the headers in an HTTP message?
Answers :- *Empty Line*

Task 5 :-HTTP Request: Request Line and Methods

An **request** is what a user sends to a web server to interact with a web application and make something happen. Since these requests are often the first point of contact between the user and the web server, knowing how they work is super important — especially if you're into cyber security.

Q11) Which HTTP protocol version became widely adopted and remains the most commonly used version for web communication, known for introducing features like persistent connections and chunked transfer encoding?
Answers :- *HTTP/1.1*

Q12) Which HTTP request method describes the communication options for the target resource, allowing clients to determine which HTTP methods are supported by the web server?
Answers :- *OPTIONS*

Q13) In an HTTP request, which component specifies the specific resource or endpoint on the web server that the client is requesting, typically appearing after the domain name in the URL?
Answers :- *URL Path*

Task 6 :-HTTP Request: Headers and Body

Request Headers

Request Headers allow extra information to be conveyed to the web server about the request. Some common headers are as follows:

Common Request Headers

Request Header

Example

Description

Host

```
Host: tryhackme.com
```

Specifies the name of the web server the request is for.

User-Agent

```
User-Agent: Mozilla/5.0
```

Shares information about the web browser the request is coming from.

Referer

```
Referer: https://www.google.com/
```

Indicates the URL from which the request came from.

Cookie

```
Cookie: user_type=student; room=introtowebapplication; room_status=in_progress
```

Information the web server previously asked the web browser to store is held in cookies.

Content-Type

```
Content-Type: application/json
```

Describes what type or format of data is in the request.

Request Body

In HTTP requests such as POST and PUT, where data is sent to the web server as opposed to requested from the web server, the data is located inside the HTTP Request Body. The formatting of the data can take many forms, but some common ones are `URL Encoded`, `Form Data`, `JSON`, or `XML`.

- **URL Encoded (application/x-www-form-urlencoded)**
  A format where data is structured in pairs of key and value where (`key=value`). Multiple pairs are separated by an (`&`) symbol, such as `key1=value1&key2=value2`. Special characters are percent-encoded.

Q14) Which HTTP request header specifies the domain name of the web server to which the request is being sent?
Answers :- *Host*

Q15) What is the default content type for form submissions in an HTTP request where the data is encoded as key=value pairs in a query string format?
Answers :- *application/x-www-form-urlencoded*

Q16) Which part of an HTTP request contains additional information like host, user agent, and content type, guiding how the web server should process the request?
Answers :- *Request Headers*

Task 7 :-HTTP Response: Status Line and Status Codes

Q17) What part of an HTTP response provides the HTTP version, status code, and a brief explanation of the response's outcome?

Answers :- S*tatus Line*

Q18) Which category of HTTP response codes indicates that the web server encountered an internal issue or is unable to fulfil the client's request?

Answers :- *Server Error Responses*

Q19) Which HTTP status code indicates that the requested resource could not be found on the web server?
Answers :- *404*

Task 8 :-HTTP Response: Headers and Body

Response Headers

When a web server responds to an request, it includes **response headers**, which are basically key-value pairs. These headers provide important info about the response and tell the client (usually the browser) how to handle it.

Picture an example of an response with the headers highlighted. Key headers like `Content-Type`, `Content-Length`, and `Date` give us important details about the response the server sends back.

Q20) Which HTTP response header can reveal information about the web server's software and version, potentially exposing it to security risks if not removed?
Answers :- *Server*

Q21) Which flag should be added to cookies in the Set-Cookie HTTP response header to ensure they are only transmitted over HTTPS, protecting them from being exposed during unencrypted transmissions?
Answers :- *Secure*

Q22) Which flag should be added to cookies in the Set-Cookie HTTP response header to prevent them from being accessed via JavaScript, thereby enhancing security against XSS attacks?
Answers :- *HttpOnly*

Task 9 :-Security Headers

Security Headers

Http Security Headers help improve the overall security of the web application by providing mitigations against attacks like Cross-Site Scripting (), clickjacking, and others. We will now dig deeper into the following security headers:

- Content-Security-Policy ()
- Strict-Transport-Security (HSTS)
- X-Content-Type-Options
- Referrer-Policy

You can use a site like https://securityheaders.io/ (opens in new tab) to analyse the security headers of any website. After the discussion in this task, you will hopefully have a better understanding of what it is reporting on.

Q23) In a Content Security Policy (CSP) configuration, which property can be set to define where scripts can be loaded from?
Answers :- *script-src*

Q24) When configuring the Strict-Transport-Security (HSTS) header to ensure that all subdomains of a site also use HTTPS, which directive should be included to apply the security policy to both the main domain and its subdomains?
Answers :- *includeSubDomains*

Q25) Which HTTP header directive is used to prevent browsers from interpreting files as a different MIME type than what is specified by the server, thereby mitigating content type sniffing attacks?
Answers :- *nosniff*

Task 10 :-Practical Task: Making HTTP Requests

That's it! You have completed all the tasks! We hope you enjoyed learning about the elements that make up web applications. Hopefully, you have learned a bit more about:

- What components are involved in web applications
- The structure of the Uniform Resource Locator (URL)
- What are messages, requests, headers and responses

- The importance of Security headers

Q25) Make a **GET** request to `/api/users`. What is the flag?

Answers :- *THM{YOU_HAVE_JUST_FOUND_THE_USER_LIST}*

Q25) Make a **POST** request to `/api/user/2` and update the **country** of Bob from **UK** to **US**. What is the flag?

Answers :- *THM{YOU_HAVE_MODIFIED_THE_USER_DATA}*

Q25) Make a **DELETE** request to `/api/user/1` to delete the user. What is the flag?

Answers :- *THM{YOU_HAVE_JUST_DELETED_A_USER}*

Q25) I'm ready to move forward and learn more about web application security.

Answers :- *No answer needed*

Happy Hacking

https://nuhhozar.blogspot.com

https://nuhhozar.medium.com

Web Applications +

Follow

Written by Nuh hozar

4 followers · 3 following

Whatever is in the name of goodness and beauty, those who know us know, those who don't know know as well as themselves.!!

No responses yet

Write a response

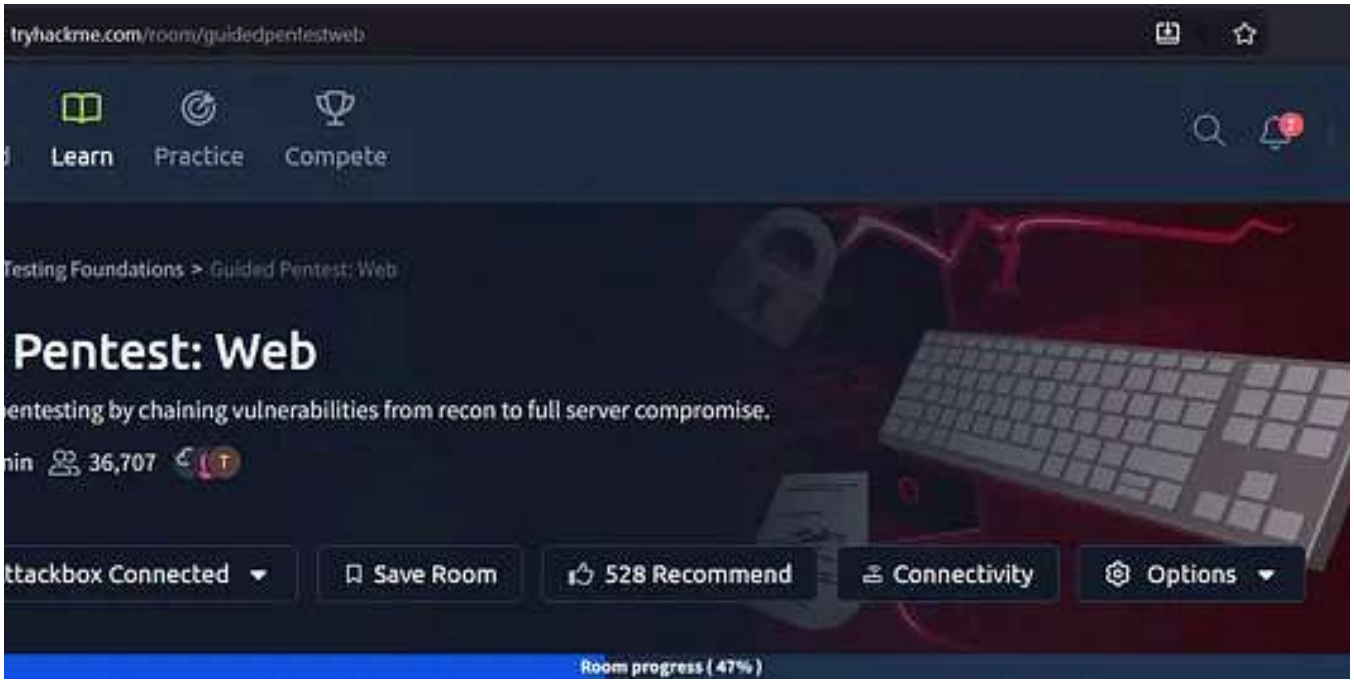
What are your thoughts?

More from Nuh hozar

Nuh hozar · Jun 25

Guided Pentest: Infrastructure Walktrough

"TRYHACKME"Guided Pentest: Infrastructure



Nuh hozar · Jun 16

Jr Penetration tester Guided Pentest Web

'' TRYHACKME '' Walkthrough

Nuh hozar · Jun 30

Tcpdump: The Basics " TRYHACKME WALKTROUGH"

Task 1- Introduction

Nuh hozar · Jul 14

JavaScript Essentials "Tryhackme Walktrough"

Task 1- Introduction

See all from Nuh hozar

Recommended from Medium